

Portable Trust Anchor for OPC UA Using Auto-Configuration

David Meier*, Florian Patzer*, Matthias Drexler* and Jürgen Beyerer*

*Fraunhofer Institute of Optonics, System Technologies, and Image Exploitation IOSB, Karlsruhe, Germany

Email: {david.meier, florian.patzer, matthias.drexler, juergen.beyerer}@iosb.fraunhofer.de

Abstract—With increasing connectivity between industrial devices their attack surface grows. Consequently, secure setups have to allow these devices to distinguish trustworthy and untrustworthy communication partners. One of the most significant and wide-spread protocols for Ethernet-based data exchange between industrial devices and controls is the Open Platform Communications Unified Architecture (OPC UA). Although the OPC UA standard includes certificate-based security measures, it lacks of applicable solutions for bootstrapping trust. For secure communication, each OPC UA application is supposed to hold an application certificate which can be managed by a Global Discovery Server (GDS). Simply put, applications request their certificate from the GDS as well as information about trustworthy and revoked certificates of third parties. However, the OPC UA specifications do not suggest a secure method to establish the initial trust for the communication between an application and the GDS. Moreover, in current implementations, the administrator has to manually interchange the certificates between the peers to build sufficient trust relationships. This paper proposes an evaluated portable trust-anchor-based concept to establish this initial trust and demonstrates it solely based on standardized OPC UA communication.

Index Terms—OPC UA, Security, Trust, Certificate, Provisioning, Trust Anchor, Global Discovery Server

I. INTRODUCTION

In today's pervasively connected industrial automation and control systems (IACS), secure communication between peers requires their mutual trust. Bootstrapping this trust is an issue researched for many years in multiple network disciplines. However, especially the industrial domain with its high heterogeneity of components and proprietary firmwares, and its multi-step assembly and manufacturing of subsystems like plant modules, is still missing sufficient solutions for this issue. Without proper trust bootstrapping solutions, applications are built which accept any new communication partner's certificates or do not even use secure communication. As an alternative, manual certificate provisioning is performed which can result in tremendous effort in provisioning and maintenance and is prone to human mistakes. It is worth noting that this is not restricted to certificates but applies to all security properties which function as trust anchors or roots of trust.

The OPC Foundation has standardized the industrial communication protocol OPC UA (Open Platform Communications Unified Architecture) which is considered to be a secure protocol, if applied with one of its secure profiles [3]. Its security relies on trust in the X.509 certificates the OPC UA

applications use for message integrity, authenticity and confidentiality. Unfortunately, the specifications lack descriptions of how to build this trust in a secure manner. This issue is well known [4].

This paper proposes a method for addressing the challenge of trust bootstrapping by utilizing a portable trust anchor. The presented concept allows a comfortable and error resistant certificate provisioning and inclusion of automated validation and on-boarding steps. The paper describes how this portable trust anchor concept can be used to bootstrap trust for standard-compliant OPC UA servers without any necessity of altering the soft- or hardware of the respective component.

The next sections are structured as follows. Section II explains the trust-related concepts of OPC UA as specified in the standard and how they are implemented in state of the art software stacks. Subsequently, section III provides a summary of research and software related to our approach. The key contribution of this paper is the trust anchor concept, which is proposed in section IV, and its practical evaluation based on OPC UA described and discussed in sections V and VI.

II. TRUST IN OPC UA

A typical protocol for modern industrial environments is given in the form of OPC UA [5], maintained by the OPC Foundation. It is a machine-to-machine network protocol specifying models for information, messages and communication. The service oriented design follows traditionally a client-server architecture, but has in the last iteration also become publisher-subscriber capable. OPC UA has security considerations built into the specification's core [6] as an integral part of various OPC UA functions and mechanisms. Most notably is the possibility to secure the communication links between server and client. For this purpose, a *SecureChannel* is defined that offers a secured and long-running connection between them. The security features can include integrity checking as well as full communication encryption. The creation of such a *SecureChannel* and the involved authentication of both sides of a communication link relies on well known X.509 certificates [7]. As shown by Fernbach and Kastner [8], OPC UA can be operated using different trust models based on these certificates. In the *direct trust* model - which is typically found in industrial networks - every OPC UA application needs to possess the certificates of each of its communications partners. While this is in its structure a very simple model, it does not scale well, as the number of certificates that need to be

exchanged rises with the number of network participants. This also leads to a demanding effort for distributing and updating these certificates manually, a process that can be error-prone. In a public *key infrastructure (PKI)* environment, applications do not need to be in possession of all certificates of potential communication partners. Instead, they are provided with certificates signed by a common certificate authority (CA). The common trust in this CA enables the applications to verify the certificates of other applications, without relying on individual approval.

OPC UA also includes specification for the management of these certificates [9]. This functionality can be provided by an OPC UA application through a *CertificateManager* as part of a *Global Discovery Server (GDS)*. The certificate management follows two possible models: *pull management* and *push management*. In the pull-management model, an OPC UA application calls methods on a GDS to request and download a signed certificate. In contrast, in push-management, the GDS calls methods on an OPC UA application to get a certificate signing request or to provide the application with a certificate. The *CertificateManager* is not only responsible for providing these so-called *ApplicationInstanceCertificates*, but also to provide additional certificate information in the form of *trust lists*. These lists can include certificates of other systems or certification bodies, the application should trust, as well as revoked certificates that should no longer be trusted.

Besides the GDS, the specification part 12 [9] also defines a Local Discovery Server (LDS). The LDS is intended to enable OPC UA clients and servers to find other servers within their subnet which provide specific OPC UA services. In the form of the *LDS with MulticastExtension (LDS-ME)*, the specification defines the support and usage of well-known auto-configuration network protocols and auto-configuration functions like *Multicast DNS (mDNS)* [10] and *DNS-Based Service Discovery (DNS-SD)* [11] to perform these functions without the need to configure any network targets manually.

III. RELATED WORK

To the best of our knowledge, no auto-provisioning method for OPC UA application certificates has been developed providing less leap of faith than the standards' GDS certificate provisioning. However, related aspects have been researched and are briefly introduced in the following paragraphs.

Papa et al. [12] describe the usage of a trust anchor to help with the establishment and preservation of trust. The authors put this trust anchor into a hardware device, which can furthermore provide additional functionality. Lesjak et al. [13] also leverage a distinct hardware module to serve as a trusted element. The authors equipped the module further with an NFC-based interface and show how this form of trust anchor can serve local, as well as remote identification and authentication. Brasser et al. [14] propose a concept which also relies on a secure integrated hardware component that even offers real-time guarantees in its operations. However, the authors do not describe how this component gets provisioned

with key material. Similarly, Rein et al. [15] describe a hardware component that acts as a "trust information agent". Their solution includes attestation of the rest of the systems, which means its state can be verified by external communication partners. But their trust establishment concept only considers the data exchange during provisioning that allows for faster verification during operation. The initial trust in the agent is still solely relying on the vendor of the hardware component.

Others researched the utilization of trusted platform modules (TPMs) for OPC UA applications for integrity attestation and key management [16] [17]. Although we agree that TPMs are useful for these tasks, this paper focuses on the OPC UA trust bootstrapping issue described in section II. For this use case, TPMs have not been applied yet. However, the method described in this paper allows a guided procedure able to include validation steps like TPM-based system integrity attestation. Moreover, cryptographic operations necessary for the here described approach may also be performed or supplemented by TPMs.

Other solutions have been developed which use certificates or keys stored on mobile USB dongles or SD cards (for simplicity we henceforth only mention dongles) and, therefore, provide sort of an alternative to actual certificate provisioning [18] [19]. Blume et al. [19] describe an approach for industrial environments, where a dongle is connected to each network participant. The network participant needs to run a specific software, which enables the device to use the USB dongle as its certificate store. This way, the certificates stored on the dongle may be used by applications on the device. Compared to the approach presented in the paper at hand, the dongle method has several disadvantages. The main issue is the necessary availability of a proprietary software on each network participant, which reduces the compatibility of the dongles to a few devices. In addition, since the dongles have to stay connected to the devices in order for the devices to work, they have to be purchased for every device, increasing its costs which might be unacceptable for smaller devices like smart sensors (not to mention that some dongles increase the device's dimensions). Moreover, failure of the dongle endangers the availability of the device's services, which again is unacceptable for critical services.

Genge et al. [20] presented a key generation scheme for end-to-end encryption, optimized for industrial use. The scheme is unfortunately relying on master keys on the devices, that need to be bootstrapped beforehand. Brost et al. of [21] also delegate the responsibility of initial trust to the vendor of the trust anchor. These publications describe secure key distribution or generation schemes but lack the initial provisioning of trust (i.e. keys or other credentials).

Bootstrapping or provisioning schemes have already been introduced for wireless sensor and similar networks [22] [23]. They are also relying on external roots of trust, like assuming trustworthy personnel [22] that needs to commission the devices initially. There are also concepts described for the deployment of trust and cryptographic key material in distributed systems. But such concepts, like the one by Kuntze

and Rudolph [24], still rely on TPMs and put their root of trust into the vendor of the TPM, who is responsible to provide initial key material during production. Other concepts, like proposed by Lee et al. [25], do not consider the initial trust establishment at all.

The fact that true initial trust establishment is an open and complex topic has also already been described in existing research. Manzoor [26] identified the "complex process of client provisioning" and Karthikeyan and Heiss [4] studied the usage of certificates in OPC UA and the challenges thereof. The authors also come to the conclusion that further study on the certificate management process of OPC UA using GDS "needs to be investigated".

This survey of the related work has shown that while there are many ways to ensure trust available, the practical process of trust provisioning is usually not really considered. This still leaves the question on how initial trust between two systems is established in the first place.

IV. PORTABLE TRUST-ANCHOR CONCEPT & REQUIREMENTS

Rotter's definition of trust as "[...] an expectancy held by an individual or a group that the word, promise, verbal or written statement of another individual or group can be relied upon." [27] is still widely accepted. However, in computer science, trust is a highly domain-specific concept which amongst others depends on the type of entity to trust in and the rules such trust is established by. Artz and Gil [28] define the type of trust we address in this paper as "Using policies to establish trust, focused on managing and exchanging credentials and enforcing access policies." and name this type *policy-based trust*. The authors further note that this trust is frequently established by using trusted third parties. Such a third party is called a *trust anchor*. A trust anchor describes a point or entity in which multiple entities root their trust in. As an example, in the case of policy-based trust, a trust anchor can serve as an authority for issuing and verifying credentials.

In this work, we present a concept that tries to achieve trust establishment as secure and usable as possible by relying on proven and standardized secure mechanisms for creating and managing trust, i.e. in the form of X.509 certificates, as well as by introducing a portable trust anchor for this purpose. Its basic principle relies on a portable device, acting as the trust anchor, that is directly connected to the target component and creates a first trust relation. It has to be noted that this connection is temporary and only needed during provisioning. Afterward, trust is securely transferred to the existing trust infrastructure of the network, for example an existing PKI.

A. Attacker Model

The portable trust anchor concept presented in this work aims at providing secure trust provisioning in networked systems. Therefore, the following attacker models were defined and used to summarize the relevant attack vectors on a trust provisioning process.

The **network-based attacker** is following the Dolev-Yao model [29]. This attacker has complete control over every network communication, enabling her or him to freely intercept, manipulate, discard or even create any data sent over the network. The network-based attacker does not have physical access to the devices attached to the network.

In contrast, the **direct attacker** is able to gain physical access to devices and the network. She or he is able to access devices directly, but is not able to perform secured functions on the system, for example functions only available to the administrator. Moreover, the direct attacker is unable to tamper with the firmware of the device itself.

Both attackers are further unable to tamper with the Main-GDS or trust anchor described in the concept.

B. Security requirements

Based on the aforementioned attacker models, the following security requirements are proposed for a secure trust establishment scheme.

SR1: Authentication A main property in the trust establishment is the connection between identities and their key material, like their public key. *Authentication* ensures this connection and makes sure that an attacker cannot impersonate another system.

SR2: Authorization Authorization ensures that only authorized changes to the trust relationships are made. If anyone, including the attacker, can establish new trust relationships, their value is nullified.

SR3: Confidentiality The established trust needs to be protected by ensuring that confidential communication stays confidential, for example by providing reliable key material to the communication partners.

SR4: Integrity The integrity of all transferred data during provisioning, for example certificate or trust data, needs to be ensured. This means that the attacker is not able to manipulate any of this data, or that any manipulation attempt can be detected by the involved parties.

SR5: Availability In many domains, like industrial automation, the availability of the systems functionality is often seen as the most important goal. Likewise, the trust anchor solution must not interfere with any given availability requirements.

C. Usability requirements

As described before, our concept's usability in real-world applications needs to be ensured for it to be adapted and properly applied and maintained. Thus the identified usability requirements are described in the following paragraphs.

UR1: Auto-configuration Configuration errors or missing security configuration are a common reason for security vulnerabilities [30]. Hence, automating the security configuration as much as possible aims at lowering the risk of bad or neglected security configurations.

UR2: Compliance A usable solution needs to adhere to the principle of compliance. It has to make sure that

implementations of the concept can follow existing standards and specifications and do not require any changes to them. This enforces the applicability of this concept in real-world environments.

UR3: Flexibility As systems and network may vary greatly in size or technology used, as seen in the different forms of industrial ethernet networks, the concept needs to be flexible enough to be applicable in most of these environments.

UR4: Scalability The mentioned growing sizes of networks and increasing numbers of networked systems, for example in the context of the growing industrial internet of things (IIoT), scalability is also a key factor for this concept.

UR5: Interoperability Similar to *Compliance*, interoperability ensures the concept keeps compatible with different technologies and is not bound to distinct implementations or products. This can be achieved, for example, by relying on well-known and accepted industry standards.

UR6: Consistency Data and trust need to remain consistent for the user of a system. The concept needs to ensure that no inconsistencies are introduced or that any inconsistency can be easily corrected.

UR7: Transparency For a solution to create trust in a networked system, transparency is key. The functional principles of the concept need to be open and during its usage the state of trust/security needs to be clear to the user.

UR8: Recoverability Well designed security solutions need to take into account possible successful attacks on the system, for example the compromise of secret key material, and offer recover strategies to regain a secured system state.

D. Portable Trust Anchor Concept

To present the portable trust anchor concept the following entities, derived from the definition in [28], are used: **credential receiver (CR)**, **credential manager (CM)** and the **portable trust anchor (PT)**. The CR is the new system or device that needs to get trust in a network of existing trust relationships. This trust relations are mainly managed by a CM, reachable for all network participants. Its functions includes, for example, updating certificates or trust lists. The CM can be a PKI system itself or can be bound to a PKI that handles the actual certificate signing. As shown in Fig. 1, the PT is only temporarily connected to the CR, specifically during the trust establishment or trust provisioning phase. It is of portable nature, which means it can be connected directly to different systems throughout the network, while it must only be connected to a single system at a time. In relation to the CR, the PT behaves similar to the CM offering *trust credentials*, i.e. signed certificates, as well as *trust information*, i.e. lists of trusted and untrusted certificates and their properties.

The concept can be divided into two main phases, as shown in Fig. 1: *trust provisioning* and *trust transfer*. For the provisioning, first the PT is connected to the CR. During the provisioning, the CR is not allowed to be connected to any other system. The CR needs then to be set into a provisioning

mode, as is common for example for newly commissioned devices. The CR then tries to find and connect to a credential manager, which will in this case be the PT. It then requests its trust credentials, from the PT. The PT - controller under administrative control - checks the request and can then issue a signed certificate. After providing the CR with the signed *initial* certificate **I** and further trust information, the physical link between CR and PT is disconnected. This concludes the trust provisioning phase. In the trust transfer phase, the CR is now connected to the network. It is able to connect to CM and authenticate itself with the signed certificate, issued by the PT, while also authenticating the CM using its trust list, also provided by the PT. This certificate is recognized by the CM, establishing trust between CR and CM. The CM can now provide the CR with a new, *operational* certificate **O** and updated trust list and thus allow secured communication with other network systems in the trust domain.

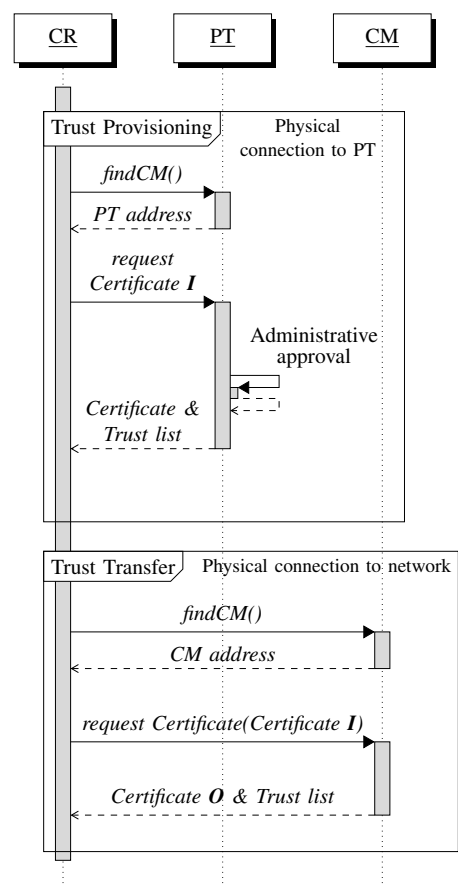


Fig. 1. Trust establishment and transfer between Credential Receiver (CR), Portable Trust Anchor (PT) and Credential Manager (CM).

It needs to be noted that during the provisioning, no administrative credentials need to be placed or entered into the CR. To clarify the trust relationships concept, Fig. 2 shows the different involved entities and their respective trust relations during provisioning and after provisioning is completed. Also note that the CM has to put *CA Trust* onto the PT, meaning it trusts the PT with issuing certificates.

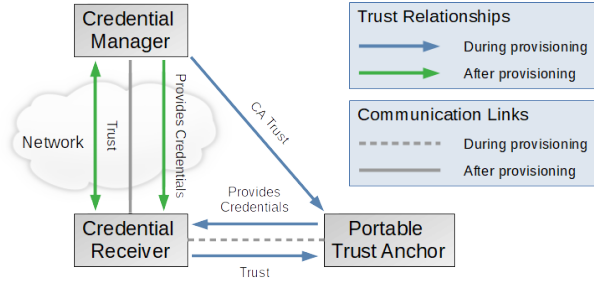


Fig. 2. Trust relationships between involved entities during and after the provisioning.

E. Concept Variations

Besides the basic form of the presented portable trust anchor concept, multiple variations of the concept are possible:

- 1) **PT as intermediate certificate authority:** Instead of being separated, the PT could also be a substitute of the CM, enabling the CR to directly start to securely communicate with other network parties, as its certificate would already be trusted by them.
- 2) **Combination of CM and PT:** If a more static trust network is used, the need for a network accessible CM might not be given. In this scenario, the PT becomes the CM for all network participants. In this case, typical credential management functions would be carried out by the PT by reusing the *Trust Provisioning* process.

V. PORTABLE TRUST ANCHOR FOR OPC UA

This section will demonstrate the trust anchor concept applied using OPC UA mechanisms and improving its functionality of trust establishment. The implementation is relying on several features of OPC UA as introduced in section II. To demonstrate the different entities involved in the concept, the following OPC UA components will be used: *Main-GDS*, *Provisioning-GDS*, *UA-Application*.

- **Main-GDS:** This is a GDS as used in a typical OPC UA network. It includes the functionality of the certificate manager to handle certificate requests. It is available to the OPC UA application via a shared network. Besides its certificate handling functions, it also serves the registration and discovery of other OPC UA applications in the network. The Main-GDS will only be responsible for already provisioned applications. It can provide these applications with updated trust lists, renewed application certificates and application discovery functions. It represents the before introduced *credential manager*.
- **Provisioning-GDS:** This is an instance of the *portable trust anchor* that will be connected to the device that needs to be provisioned. As it is only used for device provisioning and trust establishment, it only needs a subset of the functionality of a regular GDS.
- **UA-Application:** The main purpose of our trust anchor is to provide OPC UA applications with trust information, i.e. valid certificates and trust lists. The application needs

to be able to communicate and use the functions provided by GDS instances. It can rely on the two function models pull management and push management. The application represents the before introduced *credential receiver*.

Not every OPC UA application supports every possible aspect of the OPC UA specification and they do not have to. Instead, applications implement *profiles* that are defined by the standard and include subsets of all available capabilities. This concept acknowledges these slight differences, as shown in the following process descriptions.

A. Trust Provisioning

In a first step, the new device running the *UA-Application* is connected with the portable trust anchor, running the *Provisioning-GDS*. During provisioning, this is the only allowed network connection of both these systems.

After establishing the network connection to the portable trust anchor, there are multiple possible methods for the new device to connect to the *Provisioning-GDS*, depending on its capabilities. If the device supports mDNS and DNS-SD the *UA-Application* can use both to find available Global Discovery Servers, i.e. the *Provisioning-GDS* during the provisioning, and their respective addresses without the need for further configuration. This method is shown in Fig. 3. If the application lacks these capabilities, a well known GDS address needs to be configured instead.

In the next step, as defined by the OPC UA specification, the *UA-Application* needs to register itself with the GDS. After registration, the process can follow two possibilities:

- 1) The application can request a certificate from the GDS using the *pull management* model.
- 2) The GDS initializes the certificate creation process by using the *push management* model.

Either way, the application will not only be provided with the signed certificate, but also with a valid trust list, including the CA certificate of the OPC UA network.

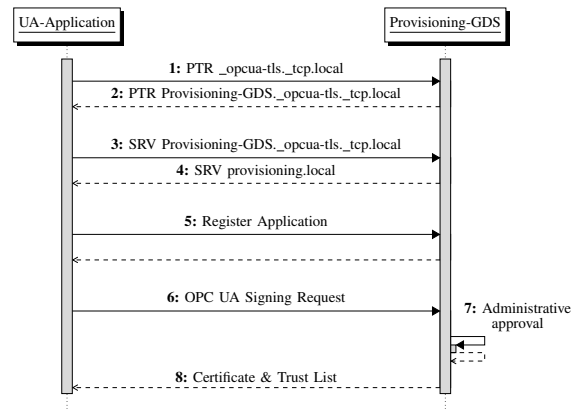


Fig. 3. Trust provisioning sequence in OPC UA using mDNS and DNS-SD.

B. Trust Transfer

The next step is the trust transfer from the *Provisioning-GDS* to the *Main-GDS*, as shown in Fig. 4. After the portable

trust anchor is disconnected from the new device, the device gets connected to the actual OPC UA network, for example via Ethernet. The application can now connect to the *Main-GDS*, which is reachable via the network, and register itself with this GDS. This is necessary, as the OPC UA specification demands the registration before the application is able to request the current trust list. The registration makes it also possible for other applications to find and get information about the new *UA-Application*. Again, depending on the GDS model deployed, the application will now request trust list updates periodically, as well as a new certificate before it expires (*pull management*), or the application will be provided with these updates on behalf of the GDS using *push management*.

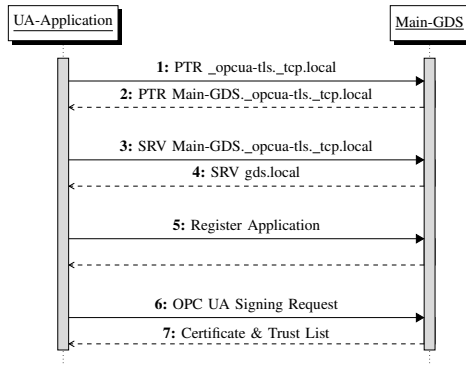


Fig. 4. Trust transfer sequence in OPC UA using mDNS and DNS-SD.

This concludes the trust transfer to the *Main-GDS* and enable the *UA-Application* to securely communicate with and authenticate other OPC UA systems in the network.

VI. EVALUATION

The portable trust anchor concept presented in this paper was evaluated by applying it to OPC UA, as described in the previous section. In this section, we will first introduce the scenario used for the evaluation. Afterwards, these results will be examined in regard to the requirements presented in section IV.

A. Evaluation Environment

The evaluation environment (cf. Fig. 5) contains a component running a standard-compliant OPC UA server (*UA-Application*) with active mDNS, which is implemented on a BeagleBoard Black. Moreover, the portable trust anchor is implemented as an OPC UA GDS (*Provisioning-GDS*) on a Raspberry Pi 3 Model B. In order to deploy an on-boarding wizard menu, the Raspberry Pi has an attached 480x320 pixel touch display. A Think Pad W530 is connected to the lab network and contains the lab network's GDS (*Main-GDS*) as well as the OPC UA client *UaExpert*¹, representing a OPC UA network. At the beginning of the scenario only the laptop is connected to the lab network. The BeagleBoard Black and the portable trust anchor run Linux-based operating systems with

¹<https://www.unified-automation.com/products/development-tools/uaexpert.html>

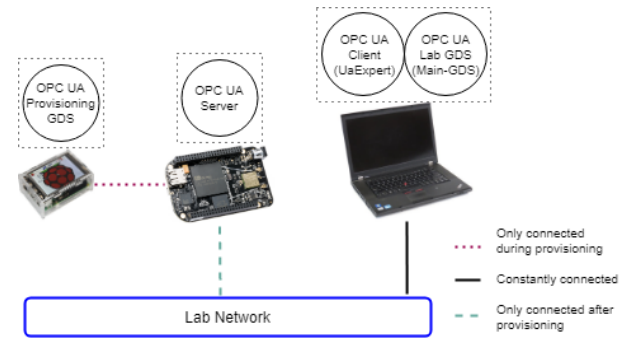


Fig. 5. Trust Anchor Evaluation Environment

dhcpcd which supports IP auto-configuration. Furthermore, both contain an active avahi-daemon which enables them to use mDNS and DNS-SD. Both GDSs are implemented using the .Net-based OPC UA reference implementation² maintained by the OPC Foundation. The portable trust anchor's hostname is set to Provisioning-GDS, whereas the Thinkpad's hostname is configured to be Main-GDS. Due to the application of DNS-SD the BeagleBoard only requests the standardized OPC UA service *_opcua-tcp._tcp.local* which will be provided by the trust anchor during the provisioning and by the Main-GDS during the operational phase. Consequently, in our setup the BeagleBoard automatically configures its network interface and learns which GDS to use. This is possible, because at one point in time, there is always only one GDS answering the service request.

B. Evaluation Scenarios

The following scenarios were evaluated using the presented evaluation environment:

- 1) Provisioning of a new device with a signed certificate and a trust list
- 2) Revocation of the Main-GDS's certificate
- 3) Revocation of the Provisioning-GDS's certificate

As initial situation, a company's CA is simply represented by a self-signed certificate which is used to sign the Provisioning-GDS certificate and the Main-GDS certificate. Furthermore, *UaExpert* and *Main-GDS* have the Provisioning-GDS's certificate in their trust list and the trust list distributed by the trust anchor contains the Main-GDS certificate. For scenario 1, the running portable trust anchor and the shutdown BeagleBoard were connected via Ethernet crossover cable. As is usual in industrial environments, the BeagleBoard starts its applications automatically after being powered on. In this case, it first performs the before described auto-configuration and starts the OPC UA server application afterwards. Subsequently, the server sends a certificate signing request to the Provisioning-GDS, which it has learned to be the GDS service provider. This contrasts with the usual implementations where the server either creates and signs its own certificate which has to be manually distributed to other applications or the alternative

²<https://github.com/OPCFoundation/UA-.NETStandard> commit da2afd8

where no certificate is created, but a signed certificate has to be installed manually before the server is started. The signing request invokes the menu of the Provisioning-GDS to notify the administrator about the signing request. At this step, the trust anchor and respectively the menu could provide additional steps like remote attestation. However, such steps are currently not part of our implementation, since they are not standardized. Thus, the administrator can simply acknowledge the certificate signing request. In consequence, the BeagleBoard is provisioned with its signed certificate and its initial trust list, containing the certificate of the Main-GDS. The last step for the administrator is to disconnect the portable trust anchor from the BeagleBoard and to connect it to the lab network. The BeagleBoard is now fully trusting the Main-GDS while also being trusted by the Main-GDS and UaExpert which have the Provisioning-GDS's certificate in their trust list.

In the following, scenarios 2 and 3 are both expecting an already successful provisioning as described in scenario 1.

In scenario 2, the Main-GDS is considered to be compromised and can no longer be trusted. Consequently, the Main-GDS's compromised certificate is added to the certificate revocation list of the Provisioning-GDS. The Provisioning-GDS can now be used to re-provision all OPC UA devices, similar to scenario 1. As the devices are still trusting the Provisioning-GDS, updating their certificates and trust lists is possible for the Provisioning-GDS. The devices do not need to reset all their trust information. A certificate for a new Main-GDS can be generated and signed, and distributed alongside by the Provisioning-GDS.

In scenario 3, now the certificate of the Provisioning-GDS needs to be revoked. It is added manually to the certificate revocation list of the Main-GDS. This implies the need to replace all certificates signed by the Provisioning-GDS's certificate. Since the Main-GDS is still trusted, this can automatically be achieved by the certificate management mechanisms, which is - this time - performed by the Main-GDS. Afterwards, the (compromised) Provisioning-GDS is not longer able to provide OPA UC devices with certificates or trust information, as long as these devices are not manually reset.

C. Attacker Scenarios

In the following, the attacker models defined in section IV are used to evaluate three critical scenarios and analyze their consequences on the established trust relations.

a) Intercepting or manipulating Provisioning: With one exception, secret credentials are not transferred during provisioning. The Key Pair Request which is sent by an application in order to not only get a certificate signed but also to let the GDS generate key material. However, the *network-based attacker* is not able to intercept the provisioning, since the target is not yet connected to the network. The *direct attacker* on the other hand is not restricted to the network. However, the direct attacker is assumed to be unable to intercept the provisioning via a direct link without being caught.

b) Adversarial Initial Provisioning: The *network-based attacker* is not able to perform the initial provisioning unless the target is connected to the network without being provisioned. However, this would break protocol following the presented concept. The *direct attacker* could potentially perform initial provisioning if he or she gets access to the target prior to its integration. However, as long as the direct attacker does not have the certificate signing keys of a trusted party, he or she could not create a certificate trusted by other OPC UA applications. Nevertheless, the *direct attacker* could add own certificates in the target's trust list and hope for it not being erased by the regular provisioning. But even if that would work, the administrator would realize that the target is already provisioned when it does not send a certificate signing request during the regular provisioning attempt.

c) Re-Provisioning: In contrast to the adversarial initial provisioning in this scenario, the target is already connected to the network. Thus, also the *network-based attacker* could try to re-provision the target. This scenario is not directly related to the portable trust anchor concept. However, it demonstrates how important it is that application certificates are only assigned to trustworthy applications. Otherwise, both attackers could achieve to act as trusted communication peers - e.g. by letting malicious apps being regularly provisioned - and not only invoke re-provisioning but also sign new malicious certificates. The portable trust anchor concept significantly improves the certainty that the assignee is trustworthy during the actual certificate assignment.

D. Evaluation Results

In section IV, security as well as usability requirements for a secure trust anchor were established. By transferring the presented trust anchor concept to OPC UA and implementing it in an evaluation environment, it could be shown that trust was established by exchanging and signing the used certificates. Furthermore, using multiple attacker scenarios, the basic security requirements **SR1**, **SR3** and **SR4** have been met. The integrated administrative approval step ensures proper authorization (**SR2**). As the portable trust anchor is not used after the provisioning (when already specified mechanisms take over, i.e. a typical GDS), availability cannot be degraded by the concept, ensuring requirement **SR5**.

The OPC UA adaptation of the concept was also able to achieve the usability requirements by being highly automated (**UR1**) and scalable (**UR4**). This also shows its compliance (**UR2**) and interoperability (**UR5**) to different technologies. No inconsistencies were introduced (**UR6**) and the overall concept is designed to omit unnecessary complexity, ensuring **UR7**. The evaluation also showed its abilities for recovery (**UR8**). However, **UR3** still needs to be proven by future implementations of the concept.

In summary, the concept proved to provide a secure trust anchor and is a clear improvement over existing, untrusted provisioning strategies. By reducing credential-based administrative provisioning, the concept even decreases possible attack vectors and the room for human mistakes.

VII. CONCLUSION & FUTURE WORK

Industrial network participants share a common issue of having to make a leap of faith throughout the network in order to build alleged trusted communication links. In the case of the industrial protocol OPC UA, an OPC UA application has to trust a GDS without having any evidence of its trustworthiness and the same accounts for the opposite direction. This paper describes a solution to overcome this issue by skipping the communication over a potentially insecure network. A portable trust anchor concept is presented enabling a direct peer to peer communication between a component, which should receive e.g. a signed certificate by a CA, and the CA itself, represented by the trust anchor. The paper proves that this concept is implementable for standard OPC UA without tampering with the component's software. Although multiple additional on-boarding steps are supported by the concept as well, the evaluation results show that it is at least a significant and important improvement of OPC-UA's implementation and usage according to the standard.

For future work, we are planning to transfer from the GDS .Net implementation to the newly OPC-Foundation-certified open source SDK open62541 in order to strengthen the foundation of the trust anchor solution for OPC UA. Furthermore, we will implement additional on-boarding steps to prove our respective claim.

REFERENCES

- [1] N. Falliere, L. O. Murchu, and E. Chien, "W32. stuxnet dossier," *White paper, Symantec Corp., Security Response*, vol. 5, no. 6, p. 29, 2011.
- [2] A. Cherepanov and R. Lipovsky, "Industroyer: Biggest threat to industrial control systems since stuxnet," *WeLiveSecurity by ESET*. Disponible en: <https://www.welivesecurity.com/2017/06/12/industroyer-biggest-threat-industrial-control-systems-since-stuxnet/> [Consultado 24/10/18], 2017.
- [3] *Sicherheitsanalyse OPC UA*, Bundesamt für Sicherheit in der Informationstechnik, Bonn, Germany, April 2016.
- [4] G. Karthikeyan and S. Heiss, "Pki and user access rights management for opc ua based applications," in *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*. [S.l.]: IEEE, 9/4/2018 - 9/7/2018, pp. 251–257.
- [5] *OPC Unified Architecture Specification Part 1: Overview and Concepts*, 1.04 ed., OPC Foundation, November 2017.
- [6] *OPC Unified Architecture Specification Part 2: Security Model*, 1.04 ed., OPC Foundation, March 2018.
- [7] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," Internet Requests for Comments, RFC Editor, RFC 5280, May 2008. [Online]. Available: <https://tools.ietf.org/rfc/rfc5280.txt>
- [8] A. Fernbach and W. Kastner, "Certificate management in opc ua applications: An evaluation of different trust models," in *ETFA 2012*. Piscataway, NJ: IEEE, 2012, pp. 1–6.
- [9] *OPC Unified Architecture Specification Part 12: Discovery and Global Services*, 1.04 ed., OPC Foundation, February 2018.
- [10] S. Cheshire and M. Krochmal, "Multicast dns," Internet Requests for Comments, RFC Editor, RFC 6762, February 2013, <http://www.rfc-editor.org/rfc/rfc6762.txt>. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc6762.txt>
- [11] —, "Dns-based service discovery," Internet Requests for Comments, RFC Editor, RFC 6763, February 2013, <http://www.rfc-editor.org/rfc/rfc6763.txt>. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc6763.txt>
- [12] S. Papa, W. Casper, and S. Nair, "Placement of trust anchors in embedded computer systems," in *2011 IEEE International Symposium on Hardware-Oriented Security and Trust*. Piscataway: IEEE, June 2011, pp. 111–116.
- [13] C. Lesjak, T. Rupprechter, J. Haid, H. Bock, and E. Brenner, "A secure hardware module and system concept for local and remote industrial embedded system identification," in *IEEE [International Conference on] Emerging Technologies and Factory Automation (ETFA), 2014*. Piscataway, NJ: IEEE, 2014, pp. 1–7.
- [14] F. Brasser, B. El Mahjoub, A.-R. Sadeghi, C. Wachsmann, and P. Koerber, "Tytan," in *Proceedings of the 52nd Annual Design Automation Conference*, Unknown, Ed. New York, NY: ACM, 2015, pp. 1–6.
- [15] A. Rein, R. Rieke, M. Jäger, N. Kuntze, and L. Coppolino, "Trust establishment in cooperating cyber-physical systems," in *Security of Industrial Control Systems and Cyber Physical Systems*, ser. Lecture Notes in Computer Science, A. Bécue, N. Cuppens-Boulahia, F. Cuppens, S. Katsikas, and C. Lambrinoudakis, Eds. Cham: Springer International Publishing and Imprint and Springer, 2016, pp. 31–47.
- [16] P. Birnstill, C. Haas, D. Hassler, and J. Beyerer, "Introducing remote attestation and hardware-based cryptography to opc ua," in *ETFA 2017*. New York: IEEE, 2017, pp. 1–8.
- [17] D. Bienhaus, L. Jäger, R. Rieke, and C. Krauß, "Gateway for industrial cyber-physical systems with hardware-based trust anchors," in *Intelligent distributed computing XIII*, ser. Studies in Computational Intelligence, I. Kutenko, C. Badica, V. Desnitsky, D. El Baz, and M. Ivanović, Eds. Cham, Switzerland: Springer, 2020, vol. 868, pp. 521–528.
- [18] S. Nepal, J. Zic, D. Liu, and J. Jang, "A mobile and portable trusted computing platform," *EURASIP Journal on Wireless Communications and Networking*, vol. 2011, no. 1, 2011.
- [19] M. Blume, N. Koch, J. Intiaz, H. Flatt, J. Jasperneite, M. Schleipen, O. Sauer, and S. Dosch, "An OPC UA based approach for dynamic-configuration of security credentials and integrating a vendor independent digital product memory," 2014.
- [20] B. Genge, P. Haller, A.-V. Duka, and H. Sándor, "A lightweight key generation scheme for end-to-end data authentication in industrial control systems," *at - Automatisierungstechnik*, vol. 67, no. 5, pp. 417–428, 2019.
- [21] G. S. Brost, M. Huber, M. Weiß, M. Protsenko, J. Schütte, and S. Wessel, "An ecosystem and iot device architecture for building trust in the industrial data space," in *CPSS'18*, D. Gollmann and J. Zhou, Eds. New York, New York: The Association for Computing Machinery, 2018, pp. 39–50.
- [22] A. Ray, J. Åkerberg, M. Gidlund, and M. Bjorkman, "Initial key distribution for industrial wireless sensor networks," in *2013 IEEE International Conference on Industrial Technology*. [Piscataway, N.J.]: IEEE, 2013, pp. 1309–1314.
- [23] S. Kwon, J. Jeong, and T. Shon, "Toward security enhanced provisioning in industrial iot systems," *Sensors (Basel, Switzerland)*, vol. 18, no. 12, 2018.
- [24] N. Kuntze and C. Rudolph, "On the automatic establishment of security relations for devices," in *Proceedings of the 13th IFIP/IEEE International Symposium on Integrated Network Management (IM 2013)*, F. De Turck, Y. Diao, and C. Hong, Eds., vol. 2013. United States of America: IEEE, Institute of Electrical and Electronics Engineers, 2013, pp. 832 – 835.
- [25] R. P. Lee, K. Markantonakis, and R. N. Akram, "Provisioning software with hardware-software binding," in *ARES 2017*. New York, New York: Association for Computing Machinery, 2017, pp. 1–9.
- [26] A. Manzoor, "Securing device connectivity in the industrial internet of things (iiot)," in *Connectivity frameworks for smart devices*, ser. Computer Communications and Networks, Z. Mahmood, Ed. Switzerland: Springer, 2016, pp. 3–22.
- [27] J. B. Rotter, "A new scale for the measurement of interpersonal trust," *Journal of personality*, vol. 35, no. 4, pp. 651–665, 1967.
- [28] D. Artz and Y. Gil, "A survey of trust in computer science and the semantic web," *Journal of Web Semantics*, vol. 5, no. 2, pp. 58–71, 2007.
- [29] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theor.*, vol. 29, no. 2, p. 198–208, Sep. 2006. [Online]. Available: <https://doi.org/10.1109/TIT.1983.1056650>
- [30] G. Nebbione and M. C. Calzarossa, "Security of iot application layer protocols: Challenges and findings," *Future Internet*, vol. 12, no. 3, p. 55, 2020.